Train API

Updated - 10-03-2026

Contents

1 Resource Links 2

2 Documentation 2

3 Data Collection / Manipulation 2

4 Architecture 3

5 Database Schema 3

6 Authentication 3

7 Testing 4

8 Generative AI Declaration 4

9 Generative AI Examples 7

1 Resource Links

- GitHub Repository
- Documentation
- Presentation

2 Documentation

Documentation is provided in a live interface rather than PDF format. The live interface can be found using the link above or **https://documentation.api.train.vulpz.com**. The different endpoints provided by the API are organised into categories in the navigation pane on the left. Each endpoint has schema definitions for its possible requests and responses. All the models representing request and response types are under the Models heading on the navigation panel and can be expanded to show their child components.

3 Data Collection / Manipulation

Due to my choice of data set I found that Generative AI would struggle with understanding and parsing it. This is likely because the data is provided by real systems that have complicated interfaces and schema types. Along with this the documentation for these systems is quite hidden and would not have been included in the models training set giving it no reference points of how to understand the formats presented.

The data was collected from a number of data sources provided by NRE (National Rail) and NWR (Network Rail) through the RDM (Rail Data Marketplace). The following resources were heavily utilised.

- Network Rail Corpus
- Network Rail Schedule
- National Rail Knowledge Base Stations
- National Rail Darwin Push Port (Archive Provided By David Wheatley)
- National Rail Darwin Timetable

Once the data was collected in its respective formats summing to 100GB python scripts provided in the **collection** folder are used to generate the SQL database needed for the API.

The first operation that must be undertook is the creation of the **stations** table. This can be done by running the Python module with the following command.

python -m collection.stations.generate

This command will generate the stations table combining data from Network Rail Corpus and the National Rail Knowledge Base Stations.

Next the timetable must be generated. The following script generates the timetable data using the National Rail Darwin Timetable for each of the stations previously populated into the database.

python -m collection.timetable.generate

Finally we have to replay the National Rail Darwin Push Port messages. The archive we use is a log of all the XML messages from Darwin, these messages must be replayed in chronological order to generate the correct database state. This is done with the final Python module.

python -m collection.pushport.replay

It would be ideal to also compile data from other source to improve the accuracy of the dataset however due to time constraints this was not possible.

4 Architecture

For the API a choice was made to use the Go programming language. This choice was made on the basis that Go is a compiled static typed language which allows it to be highly performant when compared to APIs defined in other interpreted programming languages such as Python or JavaScript.

To make development of the API in Go simpler an API framework was employed. In this case the API uses a common well documented framework called Gin. This choice means there is plenty of documentation of how to use the framework and that an LLM would likely have a good amount of training data related to the framework to infer from.

For the database a choice was made to use PostgreSQL, this choice was made after trial testing with SQLite. SQLite started to show extreme slowdown during the generation of the dataset, mainly the **stops** table, this forced the choice of technology to be reconsidered. Instead of operating as a single portable file which is programmatically controlled through a driver PostgreSQL provides a full fledged engine making it much more performant and able to handle large volumes of traffic. Along with this PostgreSQL allows for more fine grained control of data types such as the use of **VARCHAR**, reducing the overall storage usage of the database.

Docker was employed to containerise the application, this allows the application to be portable across different machines including deployment to a production environment. This allows the application to always operate as intended no matter the hosting environment.

5 Database Schema

The database uses a core schema relating **customers** to **applications** and an additional table of **administrators**. Along with this there is also the tables which store the dataset, these are organised into **stations**, **journeys** and **stops**. Each station can be related to many stops and a journey can have many stops. There is also a **stations_analysis** table which is generated during data manipulation.

6 Authentication

Endpoints in the API are organised into three different levels of authorisation.

- Administration - Covers endpoints used by administrators to perform actions such as updating an application approval status.

- Protected - Covers endpoints that the customer can use to manage their account and applications, i.e. creating new applications, refreshing application keys.
- Product - This group covers endpoints that relate to the actual end user data API provided, authentication on these endpoints does not require login and instead makes use of a Bearer token scheme.

Product endpoints rely on the Bearer token authorisation scheme, this means that an application key can be used by an autonomous user to make requests. This allows for the development of external applications which make use of our API. The Authorization header provided when making requests should follow the format **Bearer <base64("<applicationId>:<applicationKey>")>** (Swagger, 2024), an adaptation of OAuth 2.0 Bearer authorisation (Jones and Hardt, 2012).

Administration endpoints make use of login endpoint which returns a JWT (JSON Web Token) (M. B. Jones and Sakimura, 2015) which can be used for authorisation. The token is used on subsequent requests to the administration endpoints. Along with verifying the JWT the service also checks that the JWT claims **"type"** attribute is set to **administrator**, this allows the service to check that the given requester should have access to the administrative endpoints. The claims in the JWT are signed using the services private key so they can be verified as untampered.

Protected endpoints make use of the same JWT (JSON Web Token) authorisation scheme but do not check the **"type"** claim. These endpoints instead verify the validity of the token then check that the customer requesting data has authorisation to access the customer data they are requesting. This is usually that the requested data is the customers own, so the identifier in the claims matches the identifier in the URL.

To sign JWTs (JSON Web Tokens) the service uses a public private Ed25519 key pair and the EdDSA digital signature scheme. This choice was made as Ed25519 presents one of the most secure signature schemes currently adopted in wide use while still being performant for low tier hardware.

7 Testing

Testing of the application is performed using two methodologies. The first is a set of unit tests written in Go, these unit tests can be invoked using the Go CLI or through CI on GitHub Actions when a commit is made to the repository.

The unit tests have coverage over general portable functionality in the application rather than testing entire endpoints at once. This means that they can be run in isolation but also that they fail to cover as much of the code as would be desired. To aid in code coverage a Postman environment and a generated client interface are used to test the end-to-end functionality of each of the endpoints provided.

8 Generative AI Declaration

Generative AI was used to aid with the development of both the API application itself and the data manipulation scripts. The following Generative AI tools were used.

- T3 Chat (t3.chat)
- GitHub Copilot (copilot)

The T3 Chat tool allows usage of a wide range of LLM models in a chat interface which aided with the development of the application through prompts phrased as questions.

GitHub Copilot integrates a chat interface into the VSCode editor and provides "Agent" mode. In this mode the models can automatically search for context within your workspace and update files. This allows for prompt to be used which ask the AI model to make changes to the code itself.

The primary models used through these tools were Claude Sonnet 4.6 and GPT 5.3 Codex due to their high LiveBench (White et al., 2025) score and balanced token cost.

References

M. B. Jones and D. Hardt. The oauth 2.0 authorization framework: Bearer token usage. RFC 6750, October 2012. URL **https://datatracker.ietf.org/doc/html/rfc6750**. Accessed: 2026-03-10.

J. Bradley M. B. Jones and N. Sakimura. Json web token (jwt). RFC 7519, May 2015. URL **https://datatracker.ietf.org/doc/html/rfc7519**. Accessed: 2026-03-10.

Swagger. Bearer authentication, November 2024. URL **https://swagger.io/docs/specification/v3_0/authentication/bearer-authentication/**. Accessed: 2026-03-10.

Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Benjamin Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh-Agrawal, Sandeep Singh Sandha, Siddartha Venkat Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. Livebench: A challenging, contamination-free LLM benchmark. In *The Thirteenth International Conference on Learning Representations*, 2025. URL **https://livebench.ai/**. Accessed: 2026-03-10.

9 Generative AI Examples

Can you add rate limiting to these endpoints using a middleware, each applicationId which can be taken from the authorisation header as is done in the ApplicationKeyMiddleware should get a certain number of requests per minute before returning a rate limit exceeded response

station.go:1-141

Planning rate limiting middleware

I'm going to inspect the existing authentication middleware (especially `ApplicationKeyMiddleware`) and routing setup first, then I'll add a reusable per-`applicationId` rate-limit middleware and wire it to the station endpoints you shared.

Searched for files and reviewed 2 files

I found the routing and middleware wiring. Next I'll add a new middleware in authentication that limits requests per applicationId and then attach it to productRouterGroup after `ApplicationKeyMiddleware`.

Reviewed 4 files and designed rate limit middleware

I'm now editing middleware.go to add a per-application fixed-window limiter and to persist applicationId in context correctly so the limiter can identify callers consistently.

Updated middleware.go and main.go, planned integration

I've wired the limiter into the product route group and set a default cap of `120` requests/minute per applicationId. I'll run a compile check now to ensure the new middleware builds cleanly.

Verified compile errors and checked workspace for problems

Implementation is complete; I'm doing a quick pass over the edited files to confirm exact placements and then I'll summarize what changed and where.

Reviewed middleware.go and main.go, planned test output handling

Implemented per-applicationId rate limiting middleware for your API-key-protected station endpoints.

What changed

1. Added a reusable parser for the `Authorization` header in middleware.go:

- parseApplicationAuthorizationHeader(header string) (applicationId, applicationKey string, ok bool)

2. Fixed `ApplicationKeyMiddleware` context storage in middleware.go:

- It now stores the app id under authentication.ApplicationIdKey correctly:
- context.Set(ApplicationIdKey, applicationId)
- Previously it was incorrectly using the key value as the context key.

3. Added `ApplicationRateLimitMiddleware(requestsPerMinute int)` in middleware.go:

- Uses an in-memory fixed 1-minute window (go-cache).
- Limits requests per applicationId.
- Returns `429` with message `"Rate Limit Exceeded"` when exceeded.
- Falls back to extracting applicationId from the auth header if not already in context.

4. Wired middleware for your product endpoints in main.go:

- Added:
  - `const productRequestsPerMinute = 120`
  - productRouterGroup.Use(keyManager.ApplicationRateLimitMiddleware(productRequestsPerMinute))
- Applied to:
  - /station
  - /station/:stationId
  - `/stations.geojson`

Validation

- I couldn't run `go test coursework,` because the tool execution was skipped.
- I did run editor diagnostics and there are currently no reported errors.

If you want, I can next make productRequestsPerMinute configurable via environment variable (instead of hardcoded `120`).

In python I want to loop over a file and only load what is needed into memory each time, I want to iterate over my file reading up to the next delimiter `<?xml version="1.0" encoding="utf-8"?>` once I reach this I take the previous text up until this point and do some manipulation on it then move onto the next `<?xml version="1.0" encoding="utf-8"?>` delimiter in the file.

To process a large file without loading it all into memory, you can create a **generator function**. This function will read the file line by line (or in small chunks) and `yield` the accumulated text whenever it encounters your specific XML declaration delimiter.

Here is an efficient way to implement this in Python:

```python
import os

def xml_chunk_reader(file_path, delimiter='<?xml version="1.0" encoding="utf-8"?>'):
    """
    Reads a file line by line and yields chunks separated by the delimiter.
    """
    with open(file_path, "r", encoding="utf-8") as f:
        buffer = []
        for line in f:
            # Check if the line contains the delimiter
            if delimiter in line and buffer:
                # If delimiter is found and we have content,
                # split the line to handle text before the delimiter
                before, after = line.split(delimiter, 1)
                if before:
                    buffer.append(before)

                # Yield the accumulated chunk
                yield "".join(buffer)

                # Reset buffer with the start of the next chunk
                buffer = [delimiter + after]
            else:
                buffer.append(line)

        # Yield the final chunk remaining in the buffer
        if buffer:
            yield "".join(buffer)

def process_file(input_path):
    for chunk in xml_chunk_reader(input_path):
        if not chunk.strip():
            continue

        # Perform your manipulation here
        print(f"Processing chunk of size: {len(chunk)} characters")
        # Example: manipulated_data = my_xml_logic(chunk)

if __name__ == "__main__":
    # Path to your large file
    path_to_file = "large_data.xml"
    if os.path.exists(path_to_file):
        process_file(path_to_file)
```

Key Features of this Approach:

1. **Memory Efficiency**: By using `for line in f`, Python uses an internal buffer to read the file, ensuring that even multi-gigabyte files don't crash your RAM.
2. **Generator Pattern**: The `yield` keyword allows you to process one chunk at a time. The program "pauses" its state, performs your manipulation, and then resumes reading exactly where it left off.
3. **Delimiter Handling**: The logic inside the loop handles cases where the delimiter might appear in the middle of a line, ensuring that the `<?xml...?>` header stays with the new chunk rather than the end of the old one.

Mathematical Note on Complexity

If n is the total number of characters in the file and m is the average size of a chunk, the space complexity of this approach is $O(m)$ rather than $O(n)$. This makes it ideal for files where:

$$m \ll n$$

can the html be modified to load the documentation json from the application docker service

Planning URL proxy via nginx

I'll update the docs page to request `documentation` and (so it actually works from your browser) add an Nginx proxy route that forwards `documentation` to the Docker service `application:8000`. I'll inspect current `compose.yml` first to align with your latest edits.

Reviewed compose.yml and scalar.html, searched for nginx config

I found `scalar.html` still calling `localhost:8000`, so I'm switching it to a same-origin `documentation` URL and adding an Nginx config that proxies that path to `application:8000`.

Updated scalar.html and checked 3 files

Yes. Updated so the HTML now loads docs through Nginx, which proxies to the `application` Docker service.

Changes made

- `scalar.html`
  - Changed:
  - `url: "http://localhost:8000/documentation"`
  - to:
  - `url: "/documentation"`
- `compose.yml`
  - Added Nginx config mount:
  - `./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro`
- `default.conf` (new)
  - Serves `index.html` at `/`
  - Proxies `documentation` to:
  - `http://application:8000/documentation`

Why this works

Using `http://application:8000` directly in browser JS usually fails (browser can't resolve Docker service DNS).
Proxying via Nginx keeps it same-origin and lets Docker DNS resolve `application` inside the container network.

Next step

Restart nginx (and app if needed):

Edited `scalar.html` +0 -0

Then open:

- `http://localhost:8160/`

GPT-5.3-Codex • 1x

Can I make my map style `{ id: "stations-average-delay", type: "circle", source: "stations", paint: { "circle-radius": ["interpolate", ["linear"], ["get", "average-delay"], 0, 1, // value 0 → 2px radius 50, 50 // value 5000 → 20px radius], "circle-color": "#e9787e", "circle-opacity": 0.5, "circle-stroke-color": "#da1e28", "circle-stroke-width": 1, "circle-stroke-opacity": 1, } }, {` have a log curve, so the difference between smaller values is more prominent

Reasoning

Yes! MapLibre/Mapbox GL JS supports `["log10", value]` expressions. You can apply it to the interpolation input to get logarithmic scaling:

```js
{
    id: "stations-average-delay",
    type: "circle",
    source: "stations",
    paint: {
        "circle-radius": [
            "interpolate",
            ["linear"],
            ["log10", ["+", ["get", "average-delay"], 1]], // +1 avoids log(0)
            0,    1,    // log10(0+1) = 0    → 1px
            1.71, 50    // log10(50+1) ≈ 1.71 → 50px
        ],
        "circle-color": "#e9787e",
        "circle-opacity": 0.5,
        "circle-stroke-color": "#da1e28",
        "circle-stroke-width": 1,
        "circle-stroke-opacity": 1,
    }
},
```

The key idea: instead of interpolating over the raw value, you interpolate over `log10(value + 1)`. The stop keys (`0` and `1.71`) are the **log-transformed** versions of your original range (`0` to `50`).

If your delay range changes, just recalculate the upper stop key: `log10(maxValue + 1)`.